

Pengembangan Aplikasi To-Do List Sederhana Menggunakan Jetpack Compose



Tujuan Praktikum

- Pelaksanaan modul praktikum ini diarahkan untuk mencapai serangkaian tujuan pembelajaran komprehensif:
- Mengenalkan dan memantapkan pemahaman mengenai cara membuat fungsi Composable dasar untuk membangun elemen UI yang mandiri dan dapat digunakan kembali.
- Memfasilitasi penguasaan terhadap tata letak adaptif dan berurutan, mencakup komponen Column, Row, Box, serta daftar efisien LazyColumn yang menjadi fondasi dari setiap aplikasi berbasis daftar.
- Mendemonstrasikan penggunaan sistem Modifier untuk mengelola penataan gaya visual secara presisi, seperti dimensi ukuran, ruang bantalan (padding), manipulasi warna, dan bentuk geometri elemen.
- Menginstruksikan implementasi pola manajemen status (State Management) reaktif menggunakan ViewModel dan struktur data yang dapat diamati (mutableStateListOf) untuk mewujudkan prinsip pemisahan logika dari antarmuka.
- Memberikan pengalaman rekayasa perangkat lunak tingkat lanjut melalui integrasi basis data lokal menggunakan pustaka Room untuk mencapai persistensi data yang permanen.

Pengertian Jetpack Compose

Jetpack Compose adalah toolkit antarmuka pengguna deklaratif dan modern untuk mem-build aplikasi Android secara native. Compose menyederhanakan dan mempercepat proses pengembangan melalui penyediaan API Kotlin yang intuitif, membebaskan pengembang dari keharusan merancang tata letak melalui Layout Editor visual atau modifikasi kode XML. Alih-alih mendikte bagaimana antarmuka harus berubah selangkah demi selangkah, pengembang cukup mendeklarasikan seperti apa tampilan aplikasi pada suatu kondisi data tertentu. Plugin kompilator Kotlin yang tertanam di dalam arsitektur Compose kemudian bertugas memetakan dan mengubah fungsi-fungsi deklaratif tersebut menjadi elemen UI yang dirender pada layar perangkat pengguna.

Konsep Composable

Komponen paling mendasar dalam arsitektur Jetpack Compose adalah fungsi Composable. Secara sintaksis, fungsi ini dideklarasikan dengan menambahkan anotasi `@Composable` tepat sebelum kata kunci `fun` pada blok bahasa pemrograman Kotlin.



Fungsi Composable memiliki karakteristik unik di mana mereka dirancang untuk mengemisikan (memancarkan) elemen tata letak alih-alih mengembalikan suatu nilai data tradisional. Fungsi-fungsi ini beroperasi secara idempoten, artinya jika dipanggil berkali-kali dengan argumen data yang identik, mereka akan selalu menghasilkan tata letak UI yang sama persis tanpa memicu efek samping (side-effects) yang tidak terprediksi. Sistem perenderan Compose bekerja dengan mendeteksi dependensi data di dalam setiap fungsi Composable. Ketika data yang mendasari fungsi tersebut mengalami perubahan, Compose akan memicu sebuah proses yang dikenal sebagai Recomposition (Rekomposisi), di mana fungsi Composable spesifik tersebut dieksekusi ulang dengan data terbaru untuk merefleksikan pembaruan pada layar tanpa menyentuh komponen lain yang tidak terdampak.

State dan State Management Sederhana

Dalam ekosistem antarmuka reaktif, "Status" atau State merujuk pada entitas nilai apa pun yang dapat bermutasi seiring berjalannya waktu dan memengaruhi presentasi visual di layar. Manajemen status adalah proses mengendalikan bagaimana status ini disimpan, diubah, dan didistribusikan ke seluruh bagian aplikasi.

Konsep StatusDeskripsi Implementasi pada Jetpack Compose

Objek Status Tradisional (Tidak Disarankan)

Menggunakan struktur data mutable klasik seperti `ArrayList<T>` atau `mutableListOf()` murni di dalam Compose merupakan praktik yang keliru. Objek ini tidak memiliki kemampuan untuk diamati secara bawaan, sehingga perubahannya tidak akan memicu proses rekomposisi, menyebabkan pengguna melihat antarmuka dengan data yang usang.

MutableState

Pemegang status yang dapat dibaca dan ditulis sedemikian rupa sehingga Compose secara otomatis dapat melacak perubahannya. Menggunakan `mutableStateOf` memungkinkan fungsi Composable yang membaca nilai tersebut untuk secara otomatis diperbarui ketika nilai bermutasi.

Remember & RememberSaveable

Fungsi utilitas `remember` digunakan untuk menjaga nilai status agar tidak diinisialisasi ulang pada setiap siklus rekomposisi. Sementara itu, `rememberSaveable` melangkah lebih jauh dengan secara otomatis menyimpan nilai yang dapat diserialisasi ke dalam format Bundle, menjaga integritas data selama peristiwa perubahan konfigurasi sistem tingkat perangkat keras, seperti rotasi orientasi layar.

ViewModel



Kelas arsitektural yang bertugas menahan data di dalam memori saat komponen siklus hidup Android (seperti Activity) dihancurkan dan dibuat ulang. Menempatkan status, seperti `mutableStateListOf`, di dalam ViewModel memastikan bahwa operasi logika terpisah secara bersih dari fungsi UI, memfasilitasi arsitektur yang lebih solid dan teruji.

Meskipun komponen LiveData sering digunakan bersama ViewModel pada arsitektur klasik berbasis XML, ia tidak dirancang dengan mempertimbangkan Compose. Penggunaan LiveData di dalam Compose mewajibkan konversi ke format State menggunakan ekstensi `observeAsState()`. Praktik yang jauh lebih direkomendasikan untuk pengembangan modern adalah menggunakan MutableState murni untuk antarmuka sederhana atau StateFlow dari pustaka Coroutines Kotlin untuk mengelola aliran data tingkat lanjut.

Sistem Material Design

Aplikasi yang dibangun dalam modul ini akan berpedoman pada Material Design 3 (M3) atau Material You. Jetpack Compose menawarkan implementasi M3 Expressive secara bawaan, yang merupakan evolusi dari pedoman desain Google dengan berfokus pada personalisasi pengguna, tipografi yang dinamis, dan sistem pewarnaan yang responsif terhadap warna latar belakang sistem pengguna.

Sistem desain ini bertumpu pada konfigurasi kelas `MaterialTheme` yang merangkum subsistem skema warna (color scheme), tipografi, dan bentuk. Peran warna di dalam Material 3 tidak didefinisikan melalui kode heksadesimal statis (misalnya `Color(#FF0000)`), melainkan dipetakan melalui peran semantik seperti warna primer (primary), warna latar permukaan (surface), dan warna batas outline (outline). Pendekatan ini memungkinkan aplikasi bertransisi secara otomatis antara tema terang (Light Mode) dan gelap (Dark Mode) tanpa intervensi manual dari pengembang.

Tools dan Persiapan Lingkungan Pengembangan

Penyiapan lingkungan pengembangan yang komprehensif dan presisi merupakan tindakan mitigasi dini terhadap potensi galat kompilasi yang sering dialami oleh mahasiswa pemula.

Android Studio Versi Terbaru: Mahasiswa diwajibkan menggunakan piranti lunak Android Studio (versi minimum disarankan setara dengan rilis Hedgehog, Iguana, atau yang lebih baru). Versi terbaru memastikan keberadaan pratinjau interaktif UI (Interactive Preview) yang mendukung fungsi Compose secara penuh tanpa memerlukan proses build berulang.



Konfigurasi Perangkat Keras dan Emulator: Pengujian kompilasi aplikasi dapat dilakukan secara virtual menggunakan Android Virtual Device (AVD) dengan resolusi perangkat modern keras seperti Pixel 5, atau menggunakan perangkat Android fisik yang dihubungkan melalui opsi USB Debugging / Wireless Debugging.

1

Langkah Praktikum: Implementasi Bertahap Aplikasi To-Do List

Proses rekayasa perangkat lunak dalam praktikum ini dilakukan menggunakan metodologi bertahap (progressive learning), di mana setiap fungsi diisolasi berdasarkan fungsionalitas dan tanggung jawabnya sebelum digabungkan menjadi arsitektur utuh.

Langkah 1: Membuat Project Baru

Buka aplikasi Android Studio.

Navigasikan kursor dan pilih opsi "New Project".

Pada jendela pemilihan templat, pilih secara spesifik opsi "Empty Compose Activity". Pilihan ini secara otomatis akan membangun kerangka proyek yang telah disesuaikan dengan infrastruktur Jetpack Compose, tanpa menyertakan berkas tata letak XML usang.

Beri nama proyek, misalnya: TodoAppMinimalist.

Tetapkan bahasa pemrograman pada Kotlin dan Minimum SDK pada API 24.

Klik tombol Finish dan biarkan sistem Gradle mengunduh seluruh dependensi yang diperlukan.

Setelah proses sinkronisasi selesai, sistem akan menghasilkan berkas MainActivity.kt. Berkas ini mewarisi kelas ComponentActivity dan mendefinisikan blok setContent sebagai gerbang utama di mana hierarki fungsi Composable pertama kali diproyeksikan.

Langkah 2: Merumuskan Struktur Data Model (Entity)

Setiap entitas tugas dalam aplikasi memerlukan identitas dan parameter deskriptif. Implementasi ini dikonstruksi melalui deklarasi Data Class sederhana di dalam Kotlin yang dirancang untuk merepresentasikan satu unit baris tugas secara efisien.

Kode Program:



Kotlin

// Berkas: TodoItem.kt

package com.example.todoappminimalist.model

data class TodoItem(

val id: Int,

val title: String,

var isCompleted: Boolean = false

)

Penjelasan Singkat:

Deklarasi data TodoItem berfungsi sebagai cetak biru model (blueprint). Properti id berjenis integer berperan sebagai kunci identifikasi unik untuk mencegah kebingungan saat sistem menghapus atau memodifikasi tugas di tengah daftar. Parameter title merupakan deskripsi tugas berupa teks, sementara isCompleted menahan status kompleksi tugas tersebut (Selesai atau Belum Selesai), diinisialisasi dengan nilai false secara baku.

Langkah 3: Mengonstruksi Logika dengan ViewModel (State Management)

Pemisahan tanggung jawab antarmuka (UI layer) dari logika operasional (business logic layer) adalah asas arsitektur perangkat lunak yang ideal. ViewModel akan digunakan sebagai ruang terisolasi tempat manipulasi daftar tugas dieksekusi.

Menyimpan mutableStateListOf di dalam kelas ViewModel adalah pendekatan yang paling disarankan bagi pemula, karena pendekatan ini melindungi struktur daftar dari kehancuran dan reinisialisasi mendadak akibat peristiwa perombakan hierarki antarmuka secara dinamis.

Kode Program:

Kotlin

// Berkas: TodoViewModel.kt

package com.example.todoappminimalist.viewmodel



```

import androidx.compose.runtime.mutableStateListOf
import androidx.lifecycle.ViewModel
import com.example.todoappminimalist.model.TodoItem

class TodoViewModel : ViewModel() {
    // Inisialisasi daftar yang dapat diamati oleh sistem Compose
    private val _todoList = mutableStateListOf<TodoItem>()
    val todoList: List<TodoItem> get() = _todoList

    private var nextId = 1

    // Fungsi menambahkan tugas baru
    fun addTodo(title: String) {
        if (title.isNotBlank()) {
            _todoList.add(TodoItem(id = nextId++, title = title))
        }
    }

    // Fungsi memanipulasi status penyelesaian (toggle Checkbox)
    fun toggleTodo(id: Int) {
        val index = _todoList.indexOfFirst { it.id == id }
        if (index != -1) {
            val item = _todoList[index]
            // Memaksa proses rekomposisi dengan operasi penyalinan data mendalam
            _todoList[index] = item.copy(isCompleted = !item.isCompleted)
        }
    }
}

```



```
// Fungsi menghapus tugas dari daftar

fun removeTodo(id: Int) {
    _todoList.removeAll { it.id == id }
}
}

12
```

Kode Program:

Kotlin

```
// Berkas: TodoInputBar.kt

package com.example.todoappminimalist.ui.components

import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
```

@Composable

```
fun TodoInputBar(onAddClick: (String) -> Unit) {
    // Variabel status lokal untuk mencatat ketikan huruf demi huruf
    var text by remember { mutableStateOf("") }
}
```

Row(

```
    modifier = Modifier
        .fillMaxWidth()
        .padding(16.dp),
    verticalAlignment = Alignment.CenterVertically
```



```

){
    OutlinedTextField(
        value = text,
        onChange = { newText -> text = newText },
        modifier = Modifier.weight(1f),
        label = { Text("Masukkan tugas baru...") },
        singleLine = true,
        colors = OutlinedTextFieldDefaults.colors(
            focusedBorderColor = MaterialTheme.colorScheme.primary,
            unfocusedBorderColor = MaterialTheme.colorScheme.outline
        )
    )
)

Spacer(modifier = Modifier.width(8.dp))

Button(
    onClick = {
        if (text.isNotBlank()) {
            onAddClick(text) // Transmisi data ke atas (Event Hoisting)
            text = "" // Pembersihan sisa teks masukan
        }
    },
    shape = MaterialTheme.shapes.medium
){
    Text("Simpan")
}
}

```

Penjelasan Singkat:



Variabel lokal text didefinisikan menggunakan fungsi `mutableStateOf("")` dan dibalut di dalam selubung memori `remember`. Ini krusial agar pada saat pengguna mengetik dan proses recomposisi layar terjadi ratusan kali per detik, variabel teks tidak tereset kembali ke bentuk hampa (`empty string`). Desain antarmuka memosisikan elemen dalam komponen `Row`, yang merepresentasikan orientasi linier secara horizontal. Konfigurasi modifier `weight(1f)` pada kolom teks memaksa elemen tersebut untuk berekspansi memakan seluruh sisa ruang kosong pada baris, menyisakan ruang presisi yang cukup pada ujung layar khusus untuk posisi tombol "Simpan". Konsep pendelegasian aksi klik mematuhi paradigma Aliran Data Satu Arah (`Unidirectional Data Flow`).

Langkah 5: Desain Baris Representasi Item Tugas Bersama Checkbox

Komponen modul kedua bertugas mempresentasikan baris demi baris penugasan harian ke layar.

Kode Program:

Kotlin

// Berkas: `TodoItemRow.kt`

```
package com.example.todoappminimalist.ui.components
```

```
import androidx.compose.foundation.layout.*
```

```
import androidx.compose.material.icons.Icons
```

```
import androidx.compose.material.icons.filled.Delete
```

```
import androidx.compose.material3.*
```

```
import androidx.compose.runtime.Composable
```

```
import androidx.compose.ui.Alignment
```

```
import androidx.compose.ui.Modifier
```

```
import androidx.compose.ui.text.style.TextDecoration
```

```
import androidx.compose.ui.unit.dp
```

```
import com.example.todoappminimalist.model.TodoItem
```

@Composable



```

fun TodoItemRow(
    item: TodoItem,
    onCheckedChange: () -> Unit,
    onDeleteClick: () -> Unit
){
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 6.dp, horizontal = 16.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surfaceVariant
        )
    ){
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(12.dp),
            verticalAlignment = Alignment.CenterVertically
        ){
            Checkbox(
                checked = item.isCompleted,
                onCheckedChange = { onCheckedChange() }
            )

            Text(
                text = item.title,
                modifier = Modifier
                    .weight(1f)
                    .padding(start = 8.dp),
            )
        }
    }
}

```



```

        style = MaterialTheme.typography.bodyLarge.copy(
            textDecoration = if (item.isCompleted) TextDecoration.LineThrough else
TextDecoration.None
        ),
        color = if (item.isCompleted) MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha =
0.5f)
            else MaterialTheme.colorScheme.onSurface
    )

    IconButton(onClick = onDeleteClick) {
        Icon(
            imageVector = Icons.Default.Delete,
            contentDescription = "Hapus Tugas",
            tint = MaterialTheme.colorScheme.error
        )
    }
}
}
}
}
}
}
}

```

Penjelasan Singkat:

Fungsi independen ini bersifat Stateless, ia beroperasi murni berdasarkan injeksi pasif berupa data entitas tugas tunggal dan pelemparan sinyal melalui fungsi Lambda. Untuk memenuhi spesifikasi visual minimalis, struktur antarmuka dibangun di atas komponen dasar Card material desain. Implementasi rekayasa visual tingkat lanjut ditunjukkan pada bagian deklarasi Text. Apabila bendera logika boolean menandai status penugasan selesai diaktifkan (`item.isCompleted == true`), instruksi secara algoritmik memerintahkan kompilator untuk mengekstrak dan memproyeksikan dekorasi tipografi garis coret silang sentral (`TextDecoration.LineThrough`) seraya memudahkan derajat transparansi atau opasitas warnanya menjadi setengah (`alpha = 0.5f`) untuk memberi sinyal pasif fungsional kepada visual pengguna.

Langkah 6: Implementasi LazyColumn dan Integrasi Antarmuka Layar Penuh



Komponen-komponen mikro di atas harus diasimilasikan secara harmonis ke dalam arsitektur tata letak induk. Kumpulan ratusan baris data memerlukan mekanisme perenderan khusus, yaitu LazyColumn.

Berbeda dengan antarmuka penumpukan Column konvensional yang secara agresif merender seluruh elemen data dalam satu lintasan tanpa mempedulikan apakah item-item tersebut muat atau tampak dalam kapasitas visual layar pengguna, LazyColumn mengadopsi mekanisme proaktif. Mekanisme adaptif yang analog dengan RecyclerView lawas ini hanya mengeksekusi komputasi memori untuk merakit baris elemen yang benar-benar terlihat di viewport (screen bounds) pada setiap fraksi detik secara dinamis.

Kode Program:

Kotlin

// Berkas: TodoScreen.kt

```
package com.example.todoappminimalist.ui.screens

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.lifecycle.viewmodel.compose.viewModel
import com.example.todoappminimalist.ui.components.TODOInputBar
import com.example.todoappminimalist.ui.components.TODOItemRow
import com.example.todoappminimalist.viewmodel.TODOViewModel

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun TODOScreen(viewModel: TODOViewModel = viewModel()) {
```



```

Scaffold(
  topBar = {
    TopAppBar(
      title = { Text("Daftar Tugas Minimalis") },
      colors = TopAppBarDefaults.topAppBarColors(
        containerColor = MaterialTheme.colorScheme.primary,
        titleContentColor = MaterialTheme.colorScheme.onPrimary
      )
    )
  },
  containerColor = MaterialTheme.colorScheme.background
) { paddingValues ->
  Column(
    modifier = Modifier
      .fillMaxSize()
      .padding(paddingValues)
  ) {
    TodoInputBar(
      onAddClick = { newTitle ->
        viewModel.addToDo(newTitle)
      }
    )

    LazyColumn(
      modifier = Modifier.weight(1f)
    ) {
      // Konfigurasi optimal LazyColumn menggunakan key parameter
      items(
        items = viewModel.todoList,

```



Kotlin

// Berkas: MainActivity.kt

package com.example.todoappminimalist

import android.os.Bundle

import androidx.activity.ComponentActivity

import androidx.activity.compose.setContent

import androidx.compose.foundation.layout.fillMaxSize

import androidx.compose.material3.MaterialTheme

import androidx.compose.material3.Surface

import androidx.compose.ui.Modifier

import com.example.todoappminimalist.ui.screens.TODO_SCREEN

import com.example.todoappminimalist.ui.theme.TODO_APP_MINIMALIST_THEME

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            TODO_APP_MINIMALIST_THEME {  
                Surface(  
                    modifier = Modifier.fillMaxSize(),  
                    color = MaterialTheme.colorScheme.background  
                ) {  
                    TODO_SCREEN()  
                }  
            }  
        }  
    }  
}
```



}

Hasil Akhir dan Alur Penggunaan

Setelah mengeksekusi kompilasi kode perangkat lunak melalui tombol Run yang disediakan pada platform Android Studio, layar emulator fisik atau virtual akan mengkonstruksikan sebuah mahakarya antarmuka fungsional.

Aplikasi mempresentasikan desain warna yang diwariskan langsung berdasarkan algoritma token warna sistem operasi bawaan milik pengguna akhir—suatu fitur yang lahir dari integrasi Dynamic Color pada skema Material Design 3 (Android 12 ke atas). Palang atas atau TopAppBar memiliki kontras rona warna primer padat yang tegas yang memberikan sensasi struktural elegan.

Alur Pembuatan Data: Pengguna mengarahkan kursor virtual pada elemen kotak masukan tugas dan menginisiasi ketukan (tap). Animasi halus mendikte tepi bingkai formulir OutlineTextField untuk menebal, mentransmisikan warna terfokus selaras dengan sistem warna utama. Karakter teks tertuang sementara interaksi disematkan dalam sel memori Compose. Begitu komando eksekusi tombol "Simpan" dipicu, logika mengirimkan instruksi mutasi ke kelas relasional penampung data dalam modul ViewModel. Baris elemen visual anyar kemudian terpapar sebagai entitas fungsional di baris paling bawah.

Alur Interaksi Data Berbasis Ripple Effect: Sentuhan jari pada elemen visual pemicu modifikasi silang (Checkbox) menghasilkan respons pendar kilauan gelombang respons sentuhan visual spesifik (Ripple Effect) yang lazim di ekosistem Material 3. Transisi visual ini merepresentasikan kompleksitas suatu tugas, membelah tipografi judul secara memanjang horizontal menggunakan indikator dekorasi teks yang dicoret diiringi penyesuaian gradasi kekaburan. Jika batas tepian antarmuka akhir tercapai ketika pengguna mengulir layar, LazyColumn akan menyajikan efek tegangan elastis visual mekanis (Stretch Overscroll) untuk umpan balik batas limit dimensi perangkat.

Latihan Tambahan: Peningkatan Menuju Standar Produksi

Untuk memberikan dimensi praktik komprehensif bagi mahasiswa yang berkeinginan mendalami struktur skalabilitas aplikasi di dunia nyata, modul ini memfasilitasi integrasi fitur asinkron yang melengkapi kriteria penugasan esensial standar.

1. Manipulasi Pembaruan Tugas Melalui Arsitektur Dialog



Fitur esensial dalam aplikasi To-Do List adalah kapabilitas dalam menyunting tugas yang mengalami kesalahan penulisan tanpa menghapusnya terlebih dahulu. Modifikasi tekstual dilangsungkan melalui penerapan antarmuka AlertDialog melayang yang bersifat reaktif.

Penambahan Logika pada ViewModel:

Kotlin

```
fun updateTodoTitle(id: Int, newTitle: String) {  
    val index = _todoList.indexOfFirst { it.id == id }  
  
    if (index != -1 && newTitle.isNotBlank()) {  
        val item = _todoList[index]  
        _todoList[index] = item.copy(title = newTitle)  
    }  
}
```

Implementasi Komponen UI Dialog:

Arsitektur kotak dialog pada Jetpack Compose diformulasikan tidak sebagai segmen fragment konvensional, melainkan sebagai variabel operasional blok yang dirender bersyarat apabila parameter keadaannya bernilai wajar (true). Komponen AlertDialog memfasilitasi penempatan tata batas (slots) khusus untuk title, text (konten formulir), dan fiksasi elemen perintah konfirmasi navigasi bawah `confirmButton`.

Kotlin

```
// EditTodoDialog.kt  
  
import androidx.compose.material3.*  
  
import androidx.compose.runtime.*  
  
import com.example.todoappminimalist.model.TodoItem  
  
  
@Composable  
fun EditTodoDialog(  
    taskToEdit: TodoItem,
```



```

onDismissRequest: () -> Unit,
onConfirm: (String) -> Unit
){
    // Memuat teks lama sebagai nilai awal untuk disunting
    var editedText by remember { mutableStateOf(taskToEdit.title) }

```

```

AlertDialog(
    onDismissRequest = { onDismissRequest() },
    title = { Text(text = "Ubah Tugas") },
    text = {
        OutlinedTextField(
            value = editedText,
            onValueChange = { editedText = it },
            singleLine = true
        )
    },
    confirmButton = {
        TextButton(
            onClick = { onConfirm(editedText) }
        ){
            Text("Simpan Perubahan")
        }
    },
    dismissButton = {
        TextButton(
            onClick = { onDismissRequest() }
        ){
            Text("Batal")
        }
    }
)

```



```

    }
)
}

```

Integrasi operasional fungsional komponen disematkan ke dalam logika baris UI utama dengan memanfaatkan sebuah variabel perantara pendeteksi kemunculan kotak seperti `var taskToEdit by remember { mutableStateOf<TodoItem?>(null) }` yang secara pasif melahirkan layar dialog ketika ia memuat isi entitas data konkrit.

2. Implementasi Persistensi Data Permanen Melalui Room Database

Struktur data `ViewModel` yang dirancang sejauh ini sepenuhnya menduduki ruang di memori volatil acak perangkat keras. Konsekuensi fundamentalnya adalah ketika pengguna membersihkan latar belakang sistem atau aplikasi mengalami `force close`, semua kompilasi data akan mengalami destruksi permanen. Persoalan krusial di dunia nyata ini ditangani oleh perpustakaan ekosistem penyedia layanan penyimpanan data bernama `Room`.

`Room Database` adalah pustaka pilar abstraksi operasional solid yang direntangkan secara berlapis menutupi fondasi primitif basis data relasional tingkat sistem (`SQLite`). `Room` menghapus kerepotan perakitan kode sintaksis `SQLite` secara manual dan mentransformasi operasi yang rentan menjadi fungsi pemetaan kueri yang diverifikasi sistem secara statis selama rentang periode kompilasi komputasi.

Implementasi struktur basis data logis modern ini disyaratkan terbagi dalam enam babak prosedural sistematis: penambahan ekstensi kompilator, formulasi hierarki tabel, pembentukan objek instrumen akses kueri (DAO), pembentukan kelas utama penyusun pusat instansi basis data (`Database Class`), perombakan sinkronisasi aliran data `ViewModel`, dan pelaporan kolektif UI berskala reaktif.

Tahap 1: Eksekusi Pustaka Dependensi (KSP)

Injeksi penambahan modul pustaka eksternal mutlak dilakukan di `build.gradle.kts` tingkat modul. Integrasi ini melibatkan implementasi modul ekstensi perangkat bahasa kompiler tingkat tinggi Kotlin Symbol Processing (KSP) yang mendeteksi dan menerjemahkan komponen `Room` secara mandiri dalam kecepatan komputasi tertinggi.

Kotlin

plugins {

```

    id("com.google.devtools.ksp") version "1.9.22-1.0.17" // Deklarasi spesifik KSP

```



```
}
```

```
dependencies {  
    val room_version = "2.6.1"  
    implementation("androidx.room:room-runtime:$room_version")  
    implementation("androidx.room:room-ktx:$room_version")  
    ksp("androidx.room:room-compiler:$room_version")  
}
```

Tahap 2: Merancang Entitas Tabel

Tabel basis data dideklarasikan menggunakan anotasi. Setiap instance kelas model diterjemahkan sebagai deretan horizontal (row) di dalam kerangka kolom struktural.

Kotlin

```
// TodoEntity.kt
```

```
import androidx.room.Entity
```

```
import androidx.room.PrimaryKey
```

```
@Entity(tableName = "todo_table")
```

```
data class TodoEntity(  
    @PrimaryKey(autoGenerate = true)
```

```
    val id: Int = 0,
```

```
    val title: String,
```

```
    val isCompleted: Boolean = false  
)
```

Tanda `@PrimaryKey(autoGenerate = true)` membebaskan pengembang dari kewajiban menghitung alokasi indeks angka ID. Mesin basis data mengambil alih kewajiban operasional increment generik untuk mencegah duplikasi kolom kritis.

Tahap 3: Membangun Data Access Object (DAO)



Logika manipulasi struktur database ditangani secara utuh melalui perantara abstraksi yang disebut antarmuka Data Access Object (DAO). DAO mengkorelasikan baris SQL murni menuju spesifikasi struktur sintaks fungsional peranti lunak Kotlin.

Kotlin

```
// TodoDao.kt
```

```
import androidx.room.*
```

```
import kotlinx.coroutines.flow.Flow
```

@Dao

```
interface TodoDao {
```

```
    @Query("SELECT * FROM todo_table ORDER BY id DESC")
```

```
    fun getAllTodos(): Flow<List<TodoEntity>>
```

```
    @Insert(onConflict = OnConflictStrategy.REPLACE)
```

```
    suspend fun insertTodo(todo: TodoEntity)
```

```
    @Update
```

```
    suspend fun updateTodo(todo: TodoEntity)
```

```
    @Delete
```

```
    suspend fun deleteTodo(todo: TodoEntity)
```

```
}
```

Pendeklarasian hasil getAllTodos() menggunakan tipe referensi dinamis asinkron Flow<List<TodoEntity>> merupakan mahakarya krusial rekayasa. Hal ini menyebabkan antarmuka Room memantulkan gelombang data terbaru tanpa batas (streams) kapanpun kueri di modifikasi di tempat yang lain tanpa perlu diselidiki kembali secara imperatif atau sinkron oleh sistem utas perender (Main Thread).

Fungsi spesifik yang mengintervensi atau mendestruksi tata letak rekaman secara asimetris harus diberi imbuhan suspend. Indikator ini merujuk kepada utilitas kapabilitas Kotlin Coroutines



yang mengeksekusi operasi tersebut secara latar belakang untuk menghindari hambatan kinerja input-output sistem yang menghentikan animasi aplikasi.

Tahap 4: Mengonstruksikan Objek Utama Basis Data (Singleton Pattern)

Menginisialisasi basis data memerlukan biaya kinerja dan utilisasi memori yang luar biasa tinggi. Menciptakan berbagai turunan atau instance ganda dari sistem ini akan memicu penyalahgunaan ruang RAM atau pemblokiran lintas jalur operasional. Penanganan logis yang disarankan adalah merumuskan prinsip pembatasan tunggal arsitektural (Singleton Object).

Kotlin

```
// TodoDatabase.kt

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(entities = , version = 1, exportSchema = false)
abstract class TodoDatabase : RoomDatabase() {
    abstract fun todoDao(): TodoDao

    companion object {
        @Volatile
        private var INSTANCE: TodoDatabase? = null

        fun getDatabase(context: Context): TodoDatabase {
            return INSTANCE?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    TodoDatabase::class.java,
                    "todo_database"
                )
                INSTANCE = instance
            }
        }
    }
}
```



```

        ).build()

        INSTANCE = instance

        instance
    }
}
}
}
}

```

Anotasi modifikasi arsitektur memori berlapis ganda `@Volatile` memastikan bahwa alokasi referensi instansi variabel tidak pernah tersembunyi ke dalam proses pelacakan ruang simpan berfrekuensi tinggi perangkat keras mikro CPU (disebut Thread Cache). Fungsi modifikasi sinkron `synchronized(this)` mensterilkan lingkungan pembentukan parameter basis data agar pada siklus waktu sekian mikro-detik manapun saat inisiasi utas simultan menyerang, hanya diperkenankan satu konstruksi eksklusif dari alat peninggi kelas `Room.databaseBuilder()` yang disetujui untuk dikerjakan.

Tahap 5: Perubahan Fungsional ViewModel Memanfaatkan StateFlow

Prosedur implementasi reaktif `mutableStateListOf` sekarang menjadi usang karena keandalan Room Database berbasis Flow telah diinisialisasi secara mandiri. Alih-alih melakukan hosting salinan data, ViewModel sekarang hanya mereduksi instruksi untuk merangkum dan mendistribusikan saluran (streams) hasil basis data secara paralel dengan menyeberangkannya melewati mekanisme korutin StateFlow pelacak pasif.

Tabel transisi arsitektural pengelolaan data reaktif Jetpack Compose:

Paradigma Pengamatan Arsitektural	Arsitektur ViewModel Rekomendasi Analisis	Implementasi
-----------------------------------	---	--------------

Model Sinkron Sederhana	<code>mutableStateListOf<T>()</code>
-------------------------	--

Menahan state untuk aplikasi dengan siklus manipulasi langsung di tempat memori tanpa penundaan penyimpanan. Jika daftar diperbarui, list dipaksa untuk menghitung modifikasi secara linear murni.

Pola Asinkron Database Modern	<code>StateFlow<List<T>></code>
-------------------------------	---

Konstruksi tingkat lanjut ekosistem Coroutine yang tahan intervensi utas ganda secara silang multithread (multi-thread safe). Ia memiliki status dasar awalan yang tidak dihancurkan meskipun UI dipindahkan paksa ke siklus transisi (background lifecycle).



Warisan Kode Kuno (Legacy)LiveData<T>

Konstruksi pengamatan siklus usang tanpa pemahaman komprehensif Jetpack Compose; memerlukan implementasi adaptif pihak ketiga kompilator observeAsState() yang memberatkan transmisi data.

Implementasi penyusunan logika dalam TodoViewModel hasil pembaruan:

Kotlin

```
// TodoViewModel.kt
```

```
import androidx.lifecycle.ViewModel
```

```
import androidx.lifecycle.viewModelScope
```

```
import kotlinx.coroutines.Dispatchers
```

```
import kotlinx.coroutines.flow.SharingStarted
```

```
import kotlinx.coroutines.flow.StateFlow
```

```
import kotlinx.coroutines.flow.stateIn
```

```
import kotlinx.coroutines.launch
```

```
class TodoViewModel(private val dao: TodoDao) : ViewModel() {
```

```
    // Konversi aliran dingin database (Flow) menjadi aliran panas pengamatan UI (StateFlow)
```

```
    val todoList: StateFlow<List<TodoEntity>> = dao.getAllTodos()
```

```
        .stateIn(
```

```
            scope = viewModelScope,
```

```
            started = SharingStarted.WhileSubscribed(5000),
```

```
            initialValue = emptyList()
```

```
        )
```

```
    fun addTodo(title: String) {
```



```

viewModelScope.launch(Dispatchers.IO) { // Eksekusi ke utas lintas belakang IO
    dao.insertTodo(TodoEntity(title = title))
}

fun toggleTodo(item: TodoEntity) {
    viewModelScope.launch(Dispatchers.IO) {
        dao.updateTodo(item.copy(isCompleted = !item.isCompleted))
    }
}

fun removeTodo(item: TodoEntity) {
    viewModelScope.launch(Dispatchers.IO) {
        dao.deleteTodo(item)
    }
}

```

Pendelegasian fungsi asinkron ditanggung oleh ruang siklus operasional `viewModelScope.launch`. Blok `Dispatchers.IO` mengalokasikan eksekusi prosedur mutasi (Insert, Delete, Update) pada keping instruksi prosesor di belakang layar tanpa mengintervensi animasi mulus (Smooth frame-rate rendering 60FPS) dari mesin visual UI antarmuka. Pengubahan bentuk data abstraksi Flow ke konstruksi operasional penyiaran simultan `StateFlow` dijalankan mulus melewati filter spesifik komando `.stateIn`. Atribut `SharingStarted.WhileSubscribed(5000)` pada dasarnya menanggguhkan penghentian pemutusan kueri ke basis data SQLite fisik selang kurun waktu 5 detik pasca-kehilangan pengamat layar UI visual, membantu menstabilkan pergantian rotasi gawai.

Tahap 6: Merangkum Ulang Sinyal di Jetpack Compose

Langkah akhir mencakup prosedur modifikasi yang dilakukan pada tingkat arsitektural tampilan teratas. Struktur fungsional iteratif blok parameter data UI pada entitas visual perulangan `LazyColumn` di modul antarmuka dialihkan secara otomatis menggunakan alat penyadap (Collector):



Kotlin

// Di dalam TodoScreen.kt

```
import androidx.compose.runtime.collectAsState
```

@Composable

```
fun TodoScreen(viewModel: TodoViewModel = viewModel()) {
```

```
    // Memecah transmisi reaktif StateFlow dan membangun integrasi reaktif State Composable
```

```
    val list by viewModel.todoList.collectAsState()
```

```
    //... Implementasi antarmuka...
```

```
    LazyColumn(modifier = Modifier.weight(1f)) {
```

```
        items(
```

```
            items = list, // Sekarang mengambil data dari saluran pemantau UI State
```

```
            key = { task -> task.id } 
```

```
        ) { task ->
```

```
            //... pemanggilan TodoItemRow...
```

```
        }
```

```
    }
```

```
}
```

Mekanisme penarikan nilai abstrak data dengan parameter modifikasi fungsi bawaan .collectAsState() memastikan sinkronisasi fungsional Jetpack Compose terhadap variabel pemancar internal Coroutines StateFlow, mendistribusikan denyutan komando pembaruan ke dalam perenderan presentasi visual antarmuka selaras secara harmonis dengan kaidah operasional sistem arsitektur murni.

Kesimpulan

Konstruksi bertahap pembuatan aplikasi pencatatan jadwal dan Daftar Tugas harian (To-Do List) fungsional minimalis ini tidak hanya melampaui proses pembentukan kerangka elemen peranti lunak semata, melainkan mengartikulasikan evolusi rasional mendalam paradigma rekayasa teknologi sistem bergerak (Mobile Engineering) platform Android



modern. Keberhasilan penyelesaian modul perkuliahan progresif ini menyingkap tabir penundukan mekanika kompleks penyusunan tata bahasa markah (Markup) usang warisan hierarki statis arsitektur XML, dan melakukan transisi holistik secara fundamental ke bahasa tingkat abstraksi imperatif adaptif yang dipresentasikan melalui sistem Jetpack Compose dan bahasa Kotlin.

Penyusunan kode deklaratif memberdayakan metodologi rekayasa fungsional di mana pengembang memproyeksikan visual murni sesuai perwakilan status di memori, melegasikan seluruh mekanisme re-komputasi manual elemen yang berbelit pada kinerja asinkron mesin reaktor (recomposition) internal pustaka perender bawaan komponen material. Mahasiswa membedah penerapan parameter efisien pemrosesan kumpulan basis data besar dengan struktur reaktif adaptif LazyColumn, perolehan gaya warna presisi dari Dynamic Color Material 3 M3 Expressive, delegasi interaksi kontrol pada pola arsitektural Unidirectional Data Flow, hingga penanganan integrasi state di lokasi persinggahan sementara (MutableState dan penopang fungsional ekstensif siklus ViewModel).

Penerapan intervensi penyimpanan basis data transaksional skala penuh menggunakan mekanisme perpustakaan lapisan abstraksi mandiri Room yang direplikasi melalui sinkronisasi utas belakang asinkron korutin StateFlow mengelevasi keseluruhan nilai proyek prototipe simulasi kelas akademik setara dengan standarisasi spesifikasi sistem perangkat lunak skala produksi solid siap rilis di perindustrian global. Praktik rekayasa integratif berorientasi masa depan yang termaktub secara sistematis di modul praktikum ini membentangkan pijakan kokoh yang relevan dalam memperlengkapi pengembang muda bagi tantangan dinamis teknologi rekayasa mobile interaktif dekade mendatang.

